
title: The Building Blocks of Neural Networks
description: In this module, you' ll get started with PyTorch, the framework that revolutionized deep learning by making it as intuitive as writing Python code. You' ll progress from a single neuron that models linear relationships to multi-neuron networks with ...

The Building Block of Neural Networks

Getting Started with PyTorch

The Delivery Problem

- A scenario to ground the understanding of what is being built with neural networks
 - A local delivery company promises customers orders within 30 minutes
 - Deliveries are made by bike
 - The goal is to solve the problem of late deliveries (e.g., being late three times in a month)

The Delivery Decision

- A specific scenario used to model the problem:
 - A new order arrives for a delivery 7 miles away
 - The core question: "Do you take the job?"
 - The decision depends on whether the delivery can be completed in under 30 minutes
- **[Goal]** Use a neural network to solve this decision-making problem

Neural Networks for Decision Making

- Neural networks are effective at solving these types of problems if they have access to data
- The simplest model to start with is a single neuron

The Single Neuron

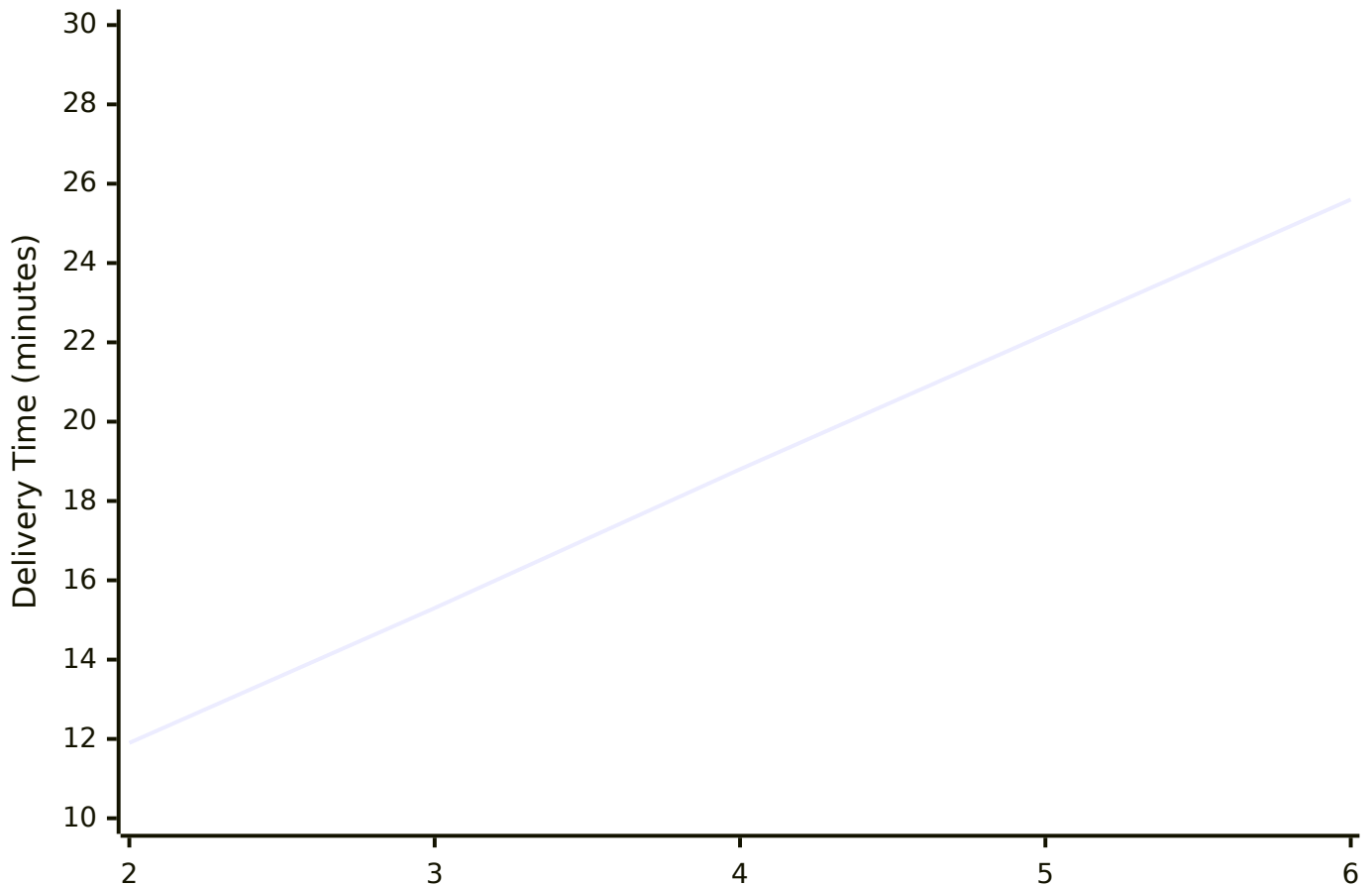
- A mathematical unit that processes information
 - Takes an **Input** and produces an **Output**
- **[Function]** It uses adjustable parameters to shift and match the patterns found in data

Delivery Data Analysis

- Historical data shows a relationship between travel distance and the time taken for delivery

Distance (miles)	Delivery Time (minutes)
2.0	11.9
3.0	15.3
4.0	18.8
5.0	22.2
6.0	25.6

Delivery Time vs Distance



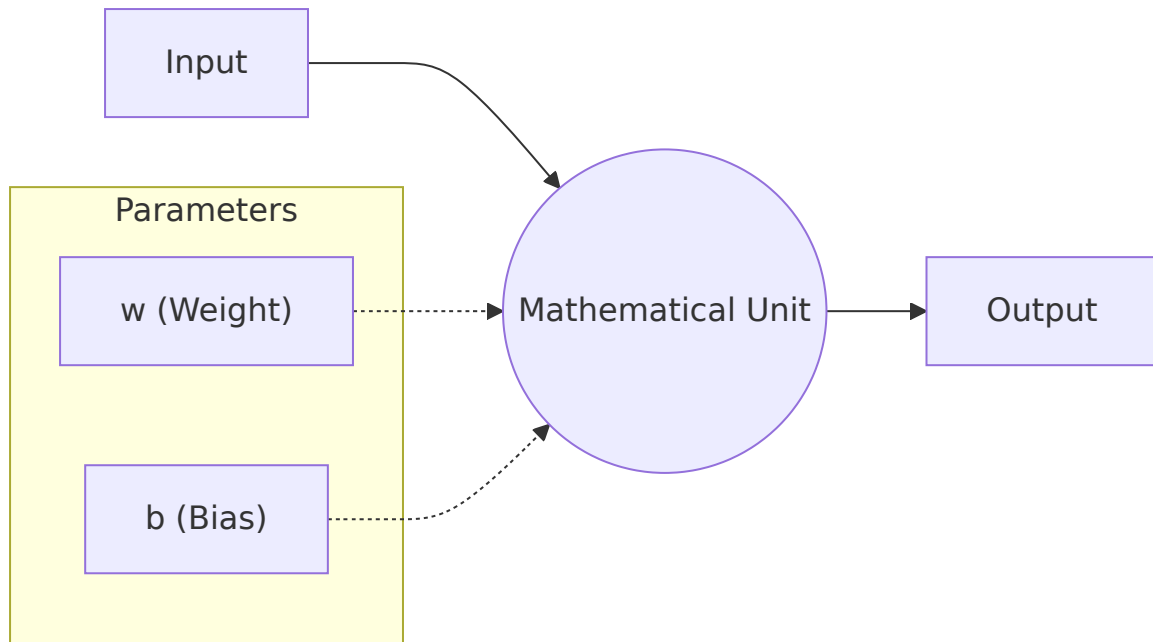
Modeling Data with a Linear Equation

- Plotting the historical delivery data reveals a clear linear pattern
 - A good predictive model for this data would be a straight line
 - If the equation for this line is known, new values (like the time for a 7-mile delivery) can be predicted
- **[Key Insight]** A single neuron is mathematically a linear equation with two parameters:
 - **Weight (w)**
 - **Bias (b)**

$$\text{time} = w \cdot \text{distance} + b$$

The Single Neuron as a Mathematical Unit

- A neuron functions as a mathematical unit that transforms an input into an output



- The operation can be expressed as:

$$\text{output} = w \cdot \text{input} + b$$

- The neuron's goal is to find the specific values for w and b that best fit the data pattern.

The Learning Process in Machine Learning

- **[Definition]** Machine learning is the process of searching for the best values for w and b to create the best-fitting line through the data

Illustrating the Search for Parameters

- To find the best line, we can start with an initial guess and compare the prediction to actual data
- **Example Guess:**
 - $w = 1.0$
 - $b = 10.0$
 - Equation: `time = 1.0 * distance + 10.0`
- **Testing the Guess (5-mile delivery):**
 - **Prediction:** $1.0 \times 5 + 10.0 = 15$ minutes
 - **Actual Data:** 22.2 minutes
 - Because the prediction (15) is far from the actual value (22.2), the current parameters are not optimal and need to be adjusted.

Refining the Parameters

- After the initial guess failed significantly (off by more than 7 minutes), the parameters were adjusted to create a steeper slope
- **Second Guess:**
 - $w = 3.4$
 - $b = 5.0$
 - Equation: `time = 3.4 * distance + 5.0`
- **Testing the New Guess (5-mile delivery):**
 - **Prediction:** 3.4
 $times5 + 5.0 = 22$ minutes
 - **Actual Data:** 22.2 minutes
 - **[Result]** The error is now only 0.2 minutes, providing a much better fit to the data.

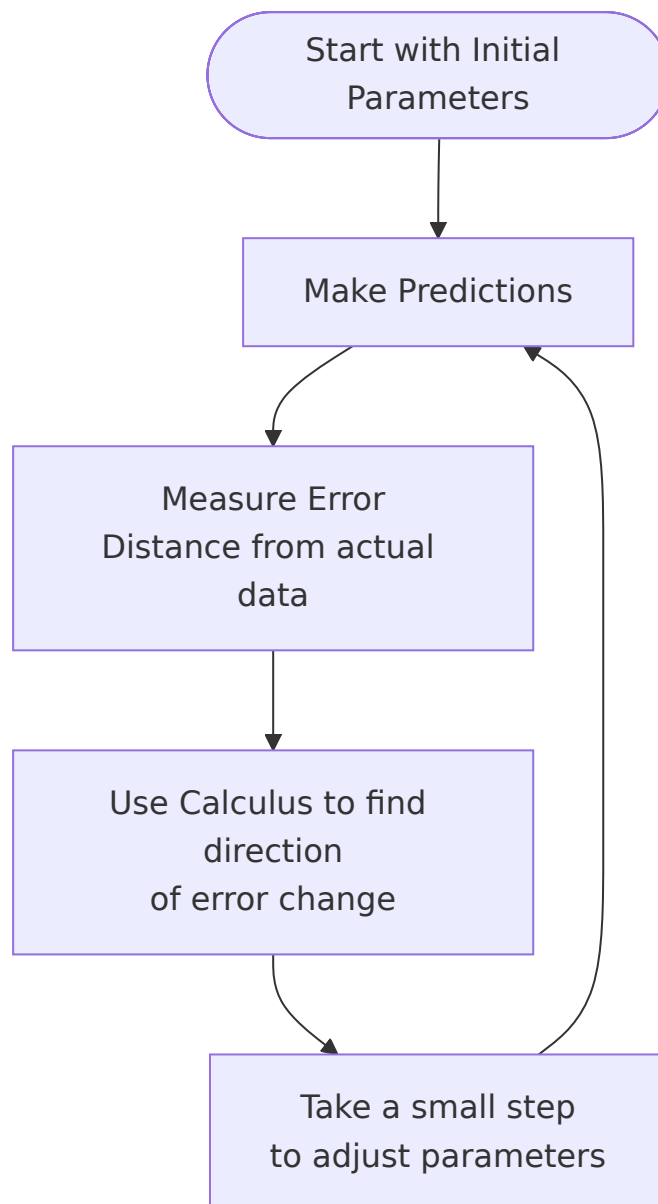
Intuition vs. Mathematics

- When a human sees a large error, they might intuitively decide to increase the weight (slope) or bias to better match the pattern
- **[Key Concept]** A neural network performs this exact same process of decision-making, but it uses mathematics instead of human intuition to determine how to adjust the parameters.

The Mathematical Optimization Process

- In PyTorch, the weight (w) and bias (b) are referred to as the model's **parameters**
- The network optimizes these parameters through an iterative cycle:
 1. **Measure Error:** The network looks at how far each prediction is from the actual data points
 - A larger distance between the prediction and the actual data results in a larger total error
 2. **Determine Direction (Calculus):** The network uses calculus to decide how to adjust the parameters
 - It essentially calculates: "If I increase the weight just a little, does the error go up or does it go down?"
 3. **Adjust Parameters:** Once the direction is known, the network takes a small step in the right direction to reduce the error

4. **Repeat:** The process of measuring, adjusting, and repeating continues until the error is minimized



The Iterative Optimization Process

- The network repeats the cycle of measuring error and adjusting parameters hundreds or even thousands of times
- **[Goal]** This repetition eventually leads to finding values that are as close as possible to the optimal w and b
- In PyTorch, this entire mathematical process is implemented with just a few lines of code

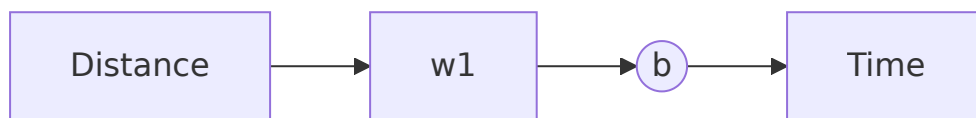
Scaling Neural Networks

- A common question is whether the math becomes impossibly complex when connecting thousands or millions of neurons

- **[Key Insight]** The complexity doesn't change the fundamental math; it stays linear
 - Each individual neuron still performs the same basic calculation regardless of the network size

Handling Multiple Inputs

- While a single neuron can work with one input (like distance), it can also be designed to handle multiple inputs



- **[Mathematical Representation]**

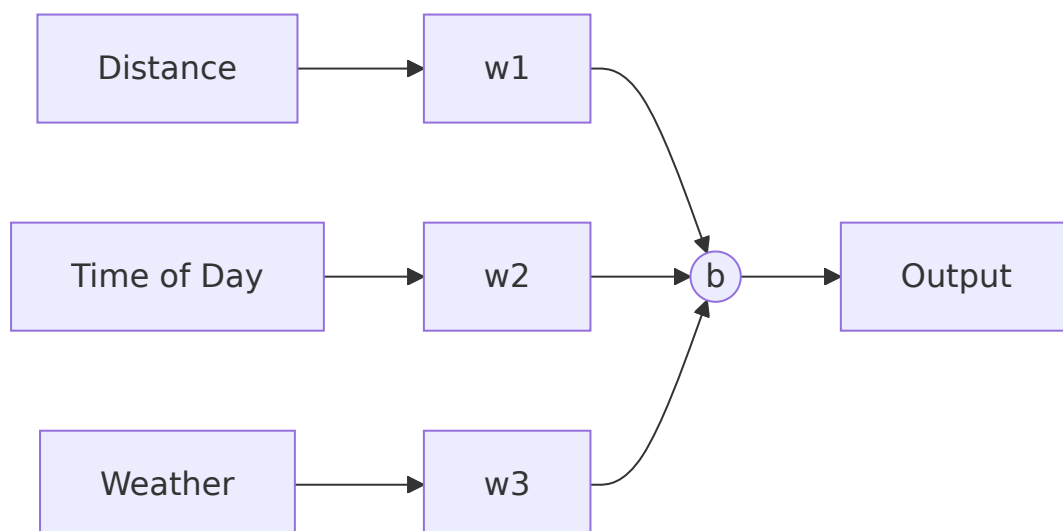
$$\text{time} = w_1 \cdot \text{distance} + b$$

Expanding to Multiple Inputs

- A neuron can process more than one factor by extending its linear equation
 - Instead of just distance, you might include variables like time of day and weather
- **[Mathematical Pattern]** Each new input is assigned its own unique weight, and they are all summed together along with a single bias value

$$\text{output} = (w_1 \cdot \text{input}_1) + (w_2 \cdot \text{input}_2) + (w_3 \cdot \text{input}_3) + b$$

- For a single neuron with multiple inputs, the equation looks like this:



Neural Network Layers

- A neural network is essentially neurons connected to other neurons

- **[Definition]** A layer is a group of neurons that all take the same set of inputs

Neural Network Architecture

A neural network is formed by connecting layers of neurons together.

- **Input Layer:** Receives the raw data (e.g., distance, time of day, weather)
- **Hidden Layers:** The layers located between the input and output layers
 - They are called "hidden" because you never directly set or see their values
- **Output Layer:** Provides the final prediction (e.g., the estimated delivery time)

Moving to PyTorch Implementation

- The theoretical concepts of neurons, layers, and optimization will be applied using PyTorch
- The next steps involve a systematic process to:
 - Build a first neural network
 - Train it using the delivery problem data
 - Transition from raw data to a deployed model

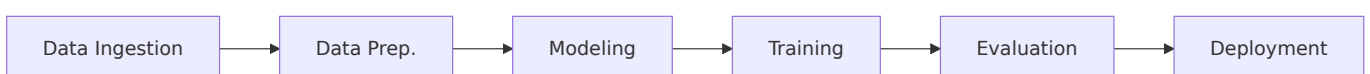
Inference

- **[Definition]** The process of using a trained model to make predictions on new, unseen data
 - In the delivery problem context, this means using the model to predict delivery times for new customer locations it has never encountered before



The ML Pipeline

Regardless of the specific application—whether predicting delivery times or classifying images—machine learning follows a consistent six-stage structure.



Data Ingestion

- The first stage of the pipeline

- **[Core Task]** Raw data collection
 - Gathering raw information
 - Organizing data so PyTorch can work with each data point efficiently

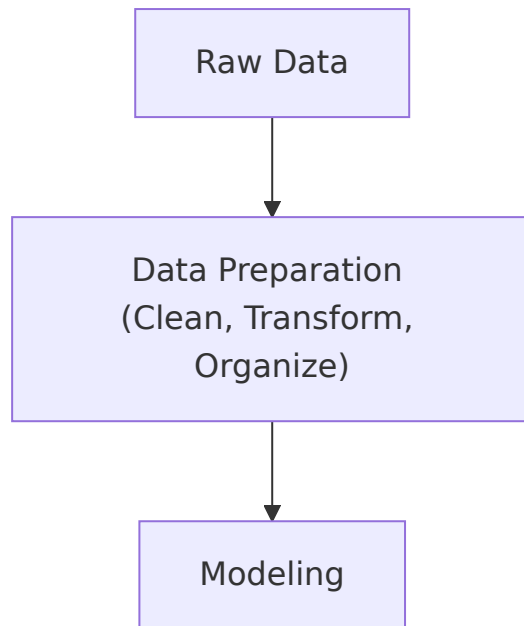
Data Ingestion Challenges

- **[The Reality of Data]** Messy data is the norm rather than the exception
- Common issues encountered:
 - **Inconsistent Formatting:** Mixing free text (e.g., "22 minutes") with numeric values (e.g., 22.0)
 - **Missing Values:** Records with "N/A" or empty fields
 - **Impossible/Outlier Values:** Negative values (e.g., -22.2) or physically improbable data (e.g., biking at 200 miles an hour)

Distance (miles)	Delivery Time (minutes)
7	"22 minutes"
7	22.0
N/A	18.8
N/A	-22.2
10 miles	3 minutes

Data Preparation

- The second stage of the pipeline
- **[Core Task]** Transforming raw data into a form a model can actually learn from
 - Cleaning
 - Transforming
 - Organizing
- **[Examples of Data Sources]**
 - Delivery records
 - Customer images
 - Email text



Data Preparation Details

- **[Core Tasks]**
 - **Fixing errors:** Removing impossible values (e.g., impossible delivery times) or duplicate entries
 - **Handling missing values:** Managing instances where data, like timestamps, were not recorded
 - **Engineering new features:** Converting raw data into more useful information (e.g., converting an address like "123 Oak Street" into a distance like "8.2 miles")
- **[The Reality of ML Development]**
 - This stage often requires the most time and the most code in a professional project
 - **[Key Insight]** Most models do not fail because the math is wrong; they fail because the data is messy

Model Building

- The third stage of the pipeline
- The third stage of the pipeline
- **[Core Task]** Designing the model architecture
 - Architecture refers to the structure of the model
 - Key decisions include:
 - How many neurons to use
 - How the neurons are connected

- What types of layers are needed

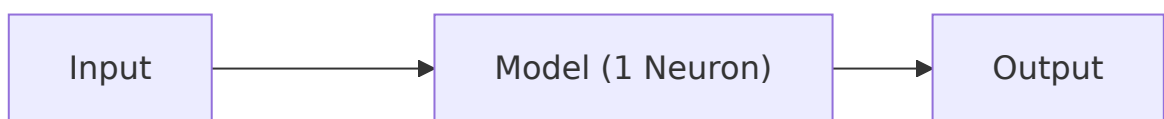
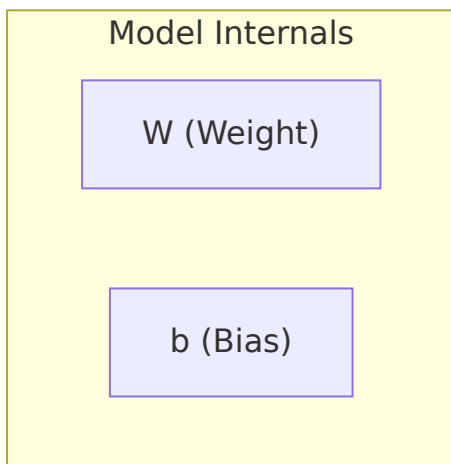
- **[Examples of Model Outputs]**

- Predicting delivery times
- Classifying images
- Analyzing text

Delivery Predictor Architecture

- In this specific example, the architecture is extremely simple
- It uses a single neuron to learn the relationship between distance and delivery time

```
model = nn.Sequential(nn.Linear(1, 1))
```



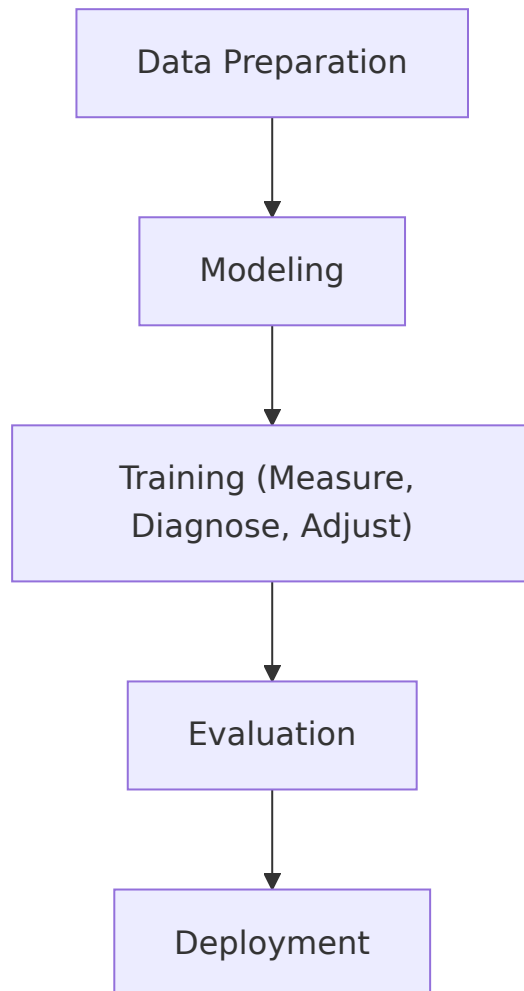
Model Building (Continuation)

- **[Implementation]** In PyTorch, defining a simple architecture can take just a single line of code
 - Even simple models follow the same fundamental patterns used in much more complex architectures

Training

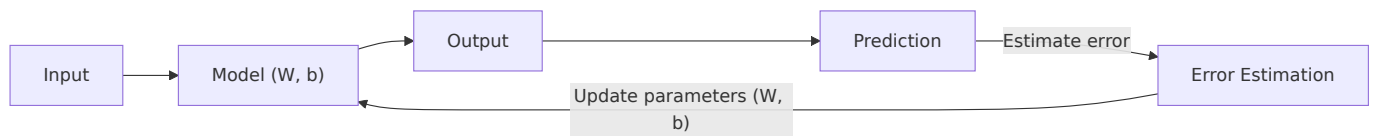
- The fourth stage of the pipeline
- **[Core Task]** Teaching the model to make accurate predictions

- **[Process]** Feeding the model examples to allow it to learn
 - Examples consist of input-output pairs
 - **[Example: Delivery Records]**
 - Input: 8.2 miles → Output: 22 minutes
 - Input: 12.5 miles → Output: 31 minutes



Training Details

- **[Configuration]** Before running the loop, you must configure key settings:
 - How to measure prediction errors
 - How to guide the model to improve
 - How fast the model learns (training settings)
- **[The Training Loop]** The iterative process where PyTorch handles the heavy lifting to update the model



Evaluation

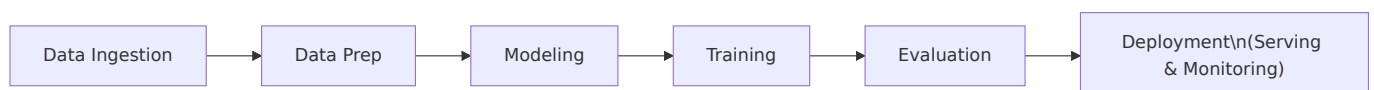
- The fifth stage of the pipeline
- **[Core Question]** Can the model make accurate predictions on new, unseen data?

Evaluation & Debugging

- **[The Goal]** To determine if a model is truly effective by seeing how it performs on new, unseen data
- **[The Test Set]** A specific portion of the data that is held back and not used during the training stage
 - This ensures the model is being tested on examples it hasn't already "seen" or learned from
- **[Measuring Performance]** Comparing model predictions against actual real-world values
 - **[Example: Delivery Prediction]**
 - Input: 15 miles
 - Model Prediction: 28 minutes
 - Actual Reality: 32 minutes
 - By analyzing these differences, you can determine how often the model is close to the truth versus how often it is significantly off
- **[Debugging]** A critical part of the process used to detect and fix deeper issues when model performance is poor

Deployment

- The final stage of the ML pipeline
- **[Core Goal]** Getting the model out into the real world so people can actually use it
- **[Key Components]**
 - Serving: Providing predictions to users
 - Monitoring: Keeping track of how the model performs in production



Building a Simple Neural Network

Import Libraries

- Core PyTorch imports required to start building and training:

```
import torch
import torch.nn as nn
import torch.optim as optim
```

- **[Library breakdown]**
 - `torch` : Provides the core functionality
 - `nn` : Contains components used for building neural networks
 - `optim` : Provides the tools necessary for training those networks

Data Ingestion vs. Data Preparation

- In professional projects, these are treated as two separate steps
- **[Data Ingestion]**
 - The initial stage of gathering raw, often "messy" data
- **[Data Preparation]**
 - The subsequent stage of refining that data for the model

Defining Data

- While real-world data requires separate ingestion and preparation stages, this example uses pre-cleaned data
- **[Tensors]** The primary way data is represented in PyTorch
 - They act as containers that store and organize data in a format models can understand
 - They are highly optimized for the mathematical operations required by neural networks

Creating Tensors

```
# Distance in miles
distances = torch.tensor([[1.0],[2.0],[3.0],[4.0]], dtype=torch.float32)

# Delivery times in minutes
times = torch.tensor([[6.96],[12.11],[16.77],[22.21]],
dtype=torch.float32)
```

- **[Structure]** The use of nested brackets (e.g., `[[...]]`) is significant
 - The outer brackets represent a subset or a specific dimension of the data

Tensor Structure: Batches and Samples

- The entire collection of data points is referred to as a **batch**
- Each inner set of brackets represents a single **sample** within that batch
- **[Example: Single Feature]**
 - If each sample only has one feature (e.g., just distance), each inner bracket contains one value
 - For four delivery records, there are four samples: `[[1.0], [2.0], [3.0], [4.0]]`
- **[Example: Multiple Features]**
 - When a sample includes multiple inputs (e.g., distance, time of day, and weather), the inner brackets expand to hold all those values

```
# Example of multiple inputs per sample
inputs = torch.tensor([[2.0, 3.0, 4.0], [6.0, 7.0, 8.0], [11.0, 12.0,
13.0], ...], dtype=torch.float32)
```

Defining Data - Multiple Inputs

- **[Feature Boundaries]** Nested brackets are critical for defining the structure of a sample
 - Even with hundreds of features, the brackets tell PyTorch exactly where one sample ends and the next begins
- **[Data Types (dtype)]** The `dtype` parameter specifies the kind of numbers being stored
 - In the example, `dtype=torch.float32` is used

- **[Why use float32?]** It is ideal for storing 32-bit floating-point numbers, which are perfect for decimal values like delivery times

```
# Distance in miles
inputs = torch.tensor([[2.0, 3.0, 4.0], [6.0, 7.0, 8.0], [11.0, 12.0, 13.0], ...], dtype=torch.float32)

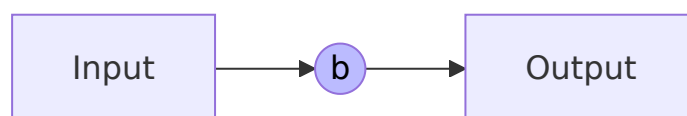
# Delivery times in minutes
times = torch.tensor([[14.8], [24.1], [36.5], [51.2], [60.8], [75.3]], dtype=torch.float32)
```

- **[The Pipeline]** Modeling is the third stage in the machine learning workflow

Building the Neural Network

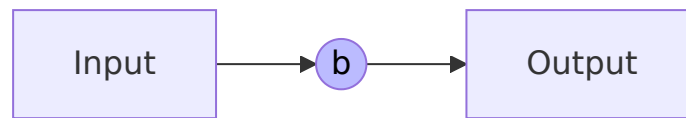
- **nn.Sequential**
 - Acts as a streamlined container for neural network layers
 - Passes data through each layer in the order they are defined
 - **[Benefit]** Makes it easy to swap out components without complex rewiring
- **nn.Linear**
 - Represents a single neuron (a linear layer)
 - **[Example: Single Input/Output]**
 - `nn.Linear(1, 1)`
 - The first parameter (1) indicates it takes one input (e.g., distance)
 - The second parameter (1) indicates it produces one output (e.g., the prediction)

```
# Define the model
model = nn.Sequential(nn.Linear(1, 1))
```



The Linear Layer Transformation

- **[Mathematical Function]** A linear layer applies a linear transformation to its inputs
 - It calculates an output using a weight (W) and a bias (b)



$$\text{Output} = W \cdot \text{Input} + b$$

- **[Learning Objective]** The neuron's goal is to learn the optimal weight and bias values
 - This process is analogous to finding the best-fitting line through a set of data points

Learning Tools

- To actually learn, a model requires two specific components:
 1. **Loss Function:** Measures how far off the model's predictions are from the actual values
 2. **Optimizer:** The mechanism used to update the model's parameters to reduce the loss

Loss Function & Optimizer

- **Mean Squared Error (MSE) Loss**
 - Used to quantify the error between predictions and targets
- **Stochastic Gradient Descent (SGD) Optimizer**
 - Used to update the model's parameters
 - Includes a learning rate (`lr`) to control the step size of updates

```
# Define the loss function and optimizer
loss_function = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

SGD Optimizer Details

- **Stochastic Gradient Descent (SGD)**
 - The algorithm responsible for determining the direction in which to adjust the weight (W) and bias (b) to reduce error
- `model.parameters()`

- A PyTorch method used to access the specific values (like W and b) that the optimizer needs to update
- **Learning Rate (lr)**
 - A hyperparameter that controls the size of the steps taken during the update process
 - **[Note]** Smaller values result in smaller, more precise adjustments

The Training Loop

- This is the process where the actual learning occurs
- It automates the iterative adjustments of parameters to minimize loss

Training Loop Steps

1. **Reset the optimizer:** Clear previous gradients
2. **Make predictions:** Generate outputs using the current model and inputs
3. **Calculate the loss:** Determine how far predictions are from the targets
4. **Calculate adjustments:** Perform backpropagation to find necessary changes
5. **Update the model:** Use the optimizer to apply the adjustments

```
# Training loop
for epoch in range(500):

    # 0. Reset the optimizer
    optimizer.zero_grad()

    # 1. Make predictions
    outputs = model(inputs)

    # 2. Calculate the loss - how bad was this guess?
    loss = loss_function(outputs, times)

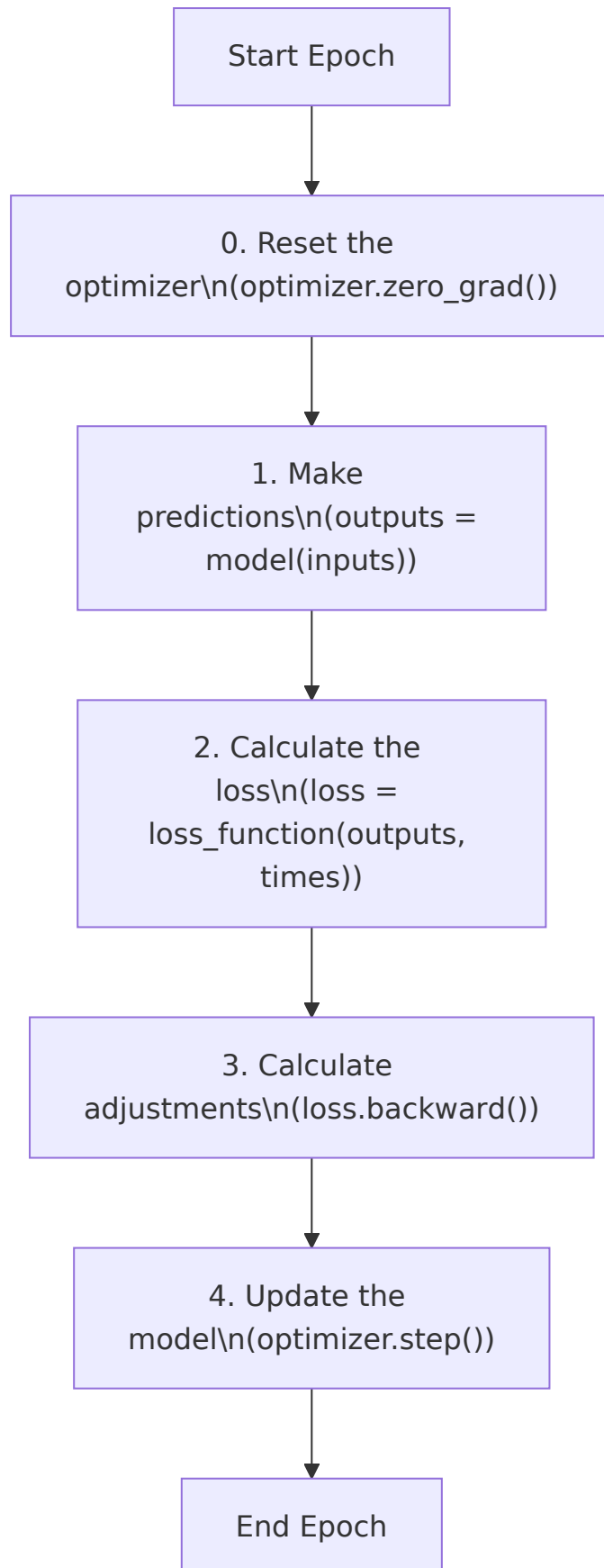
    # 3. Calculate adjustments
    loss.backward()

    # 4. Update the model
    optimizer.step()
```

Epochs and Iterative Learning

- **Epoch:** One full pass through the entire training dataset
- The training loop automates the process of adjusting parameters hundreds of times (e.g., 500 epochs) to minimize error.

Steps in an Epoch



- **0. Reset the optimizer**

- `optimizer.zero_grad()`
- Clears out previous gradients to ensure they don't interfere with the current update

- **1. Make predictions**
 - Generates outputs based on the current model state and input data
- **2. Calculate the loss**
 - Measures how far the predictions are from the actual targets
- **3. Calculate adjustments**
 - Uses backpropagation to determine how the parameters need to change
- **4. Update the model**
 - Applies the calculated adjustments to the model's parameters

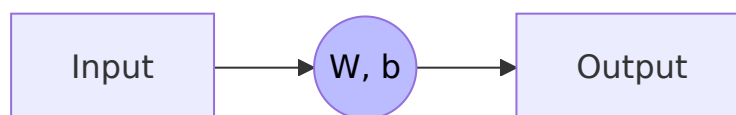
Backpropagation

- The technical process behind adjusting the weight and bias to reduce error
- It uses calculus to figure out how to modify internal parameters so that predictions become more accurate over time

Inference

- The process of using a trained model to make actual predictions on new data
- **[Why use `torch.no_grad()` ?]** Because training requires extra work under the hood to track gradients for adjustments; during inference, we skip this to run the model much more efficiently

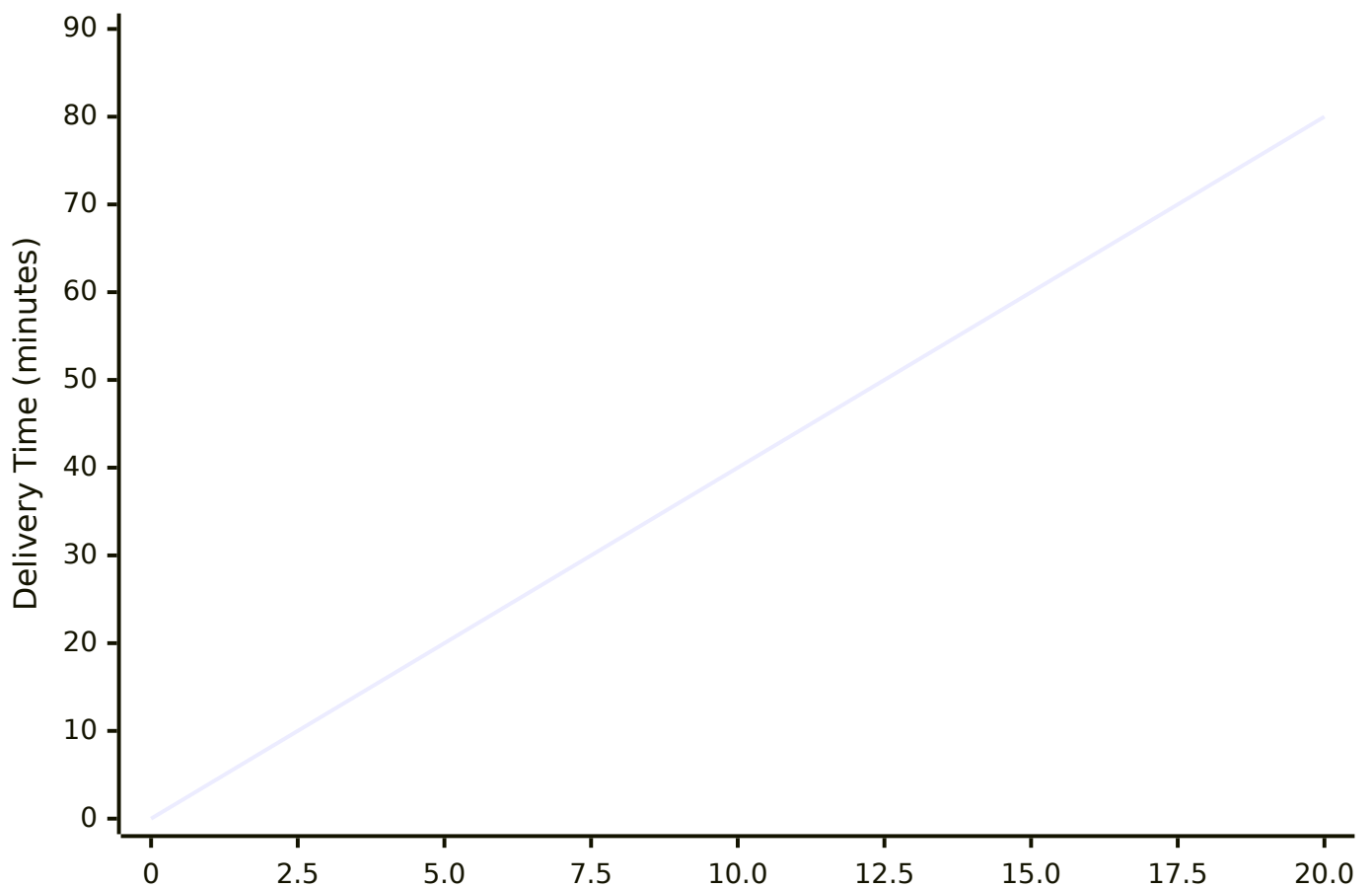
```
with torch.no_grad():  
    test_distance = torch.tensor([[25.0]], dtype=torch.float32)  
    predicted_time = model(test_distance)  
    print(f"Predicted time for 25 miles: {predicted_time.item():.1f}  
minutes")
```



- Moving from training the model to using it for real-world predictions
- This involves applying the learned weights and biases to new, unseen data
- The transition from bike-only service to citywide car operations revealed a flaw in the previous model
- **[The Problem]** As distances increased, delivery times stopped following a straight line

- **[Why the linear model failed]** It relied on a faulty assumption: that every mile requires the same amount of time

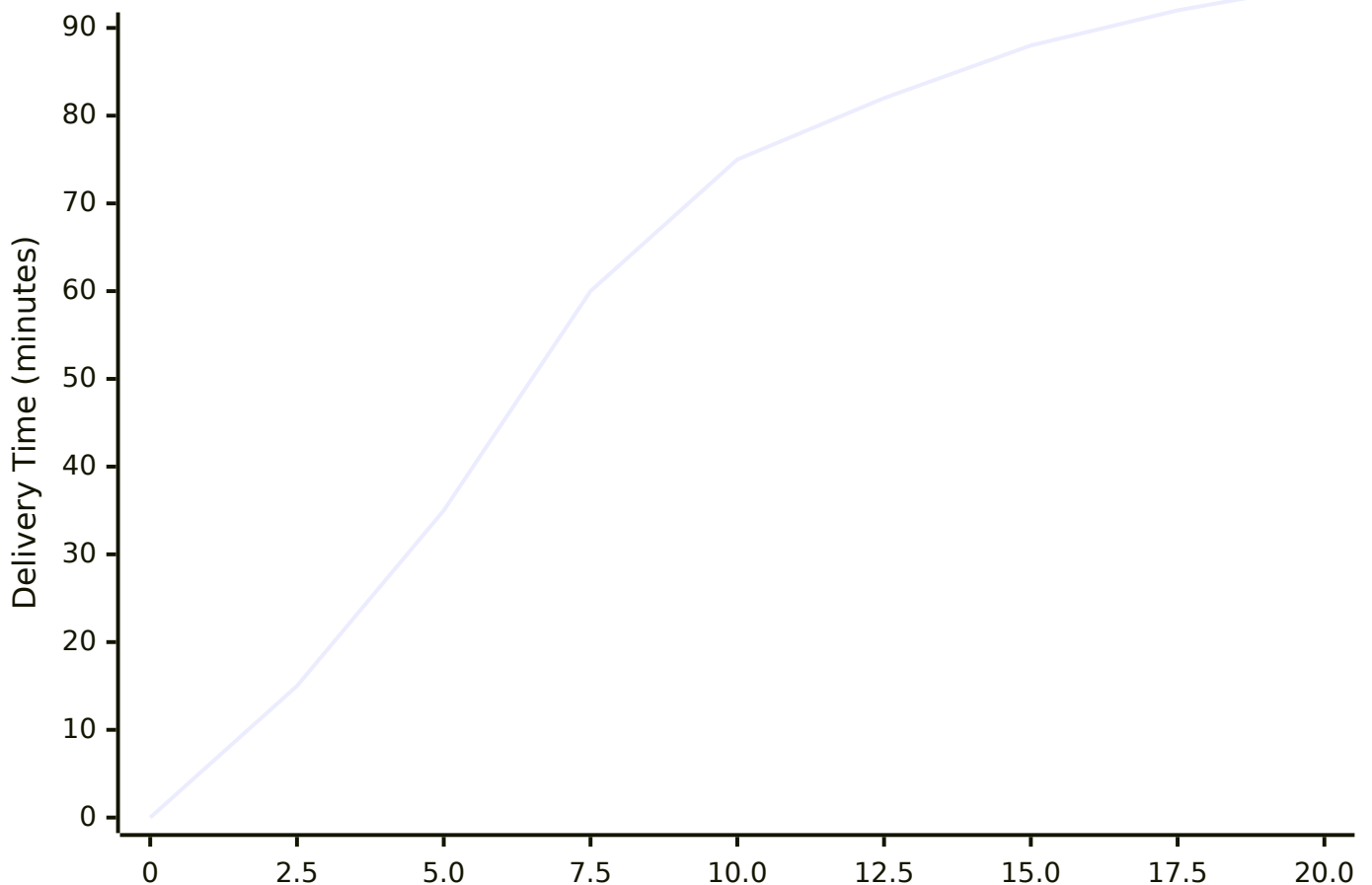
Delivery Time vs Distance (Linear Assumption)



The Non-Linear Reality

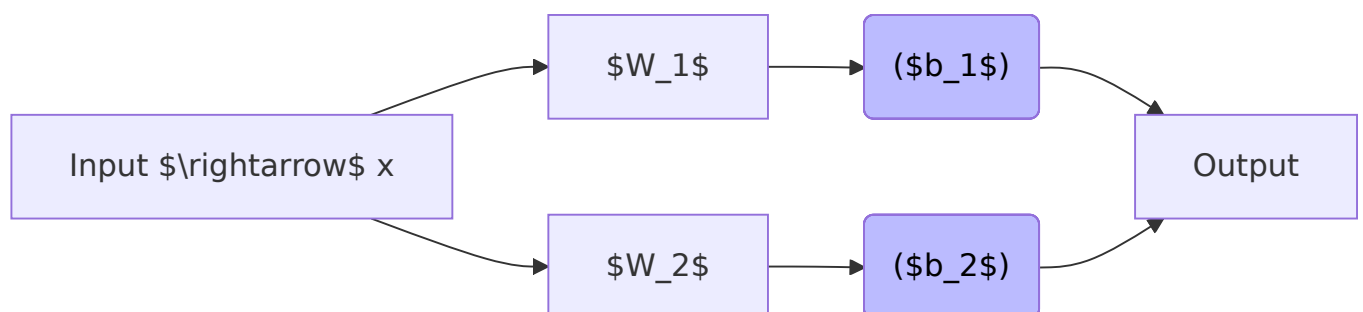
- The actual relationship between distance and delivery time is a curve, not a straight line
 - Short distances: Slowed by dense city traffic
 - Long distances: Faster as cars reach highways

Actual Delivery Time vs Distance (Non-Linear)



Moving Beyond Single Neurons

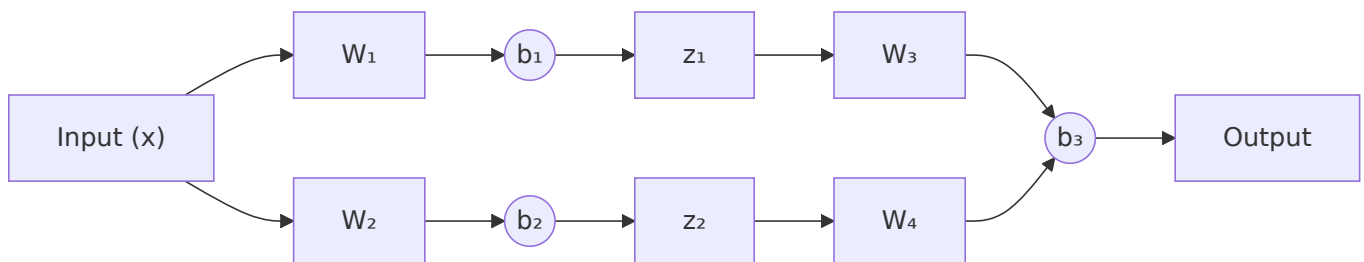
- **[The Idea]** Adding more neurons to capture more complex relationships
- **The Structural Challenge**
 - A single neuron acts as the output layer because it produces one final prediction
 - Adding a second neuron creates two separate outputs from the same input
 - To get a single final prediction, you need an additional step to combine these outputs



Adding Neurons and Hidden Layers

- To capture more complexity, we can add more neurons to the network

- **[The Hidden Layer]** When we add neurons between the input and the final output, they are considered a hidden layer
 - These neurons independently calculate a weighted sum from the input
 - For example, if we have two neurons, their intermediate outputs can be labeled z_1 and z_2
- **[The Output Neuron]** A final neuron is then used to combine these intermediate outputs into a single prediction



The Limitation of Linear Stacking

- Simply stacking neurons does not automatically allow the model to learn non-linear patterns
- **[Why?]** If every operation is just multiplying by a weight and adding a bias, the entire sequence mathematically collapses back into a single linear equation

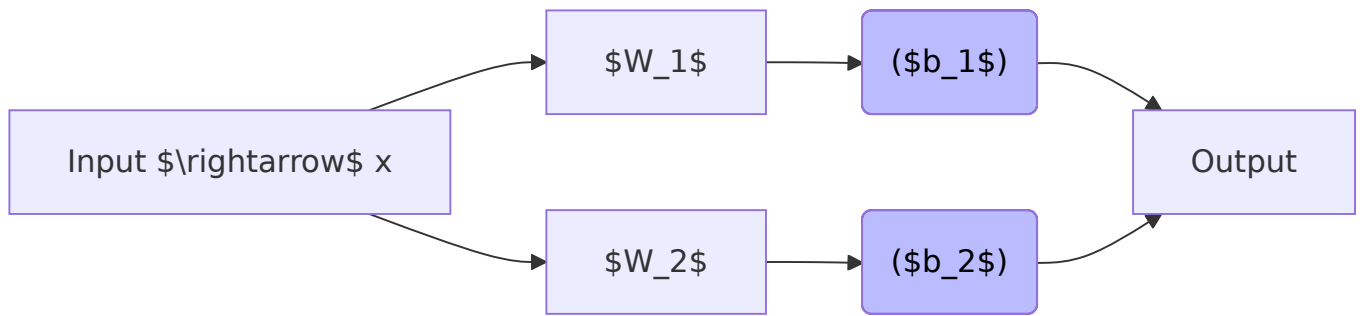
$$\text{Output} = w_3(w_1x + b_1) + w_4(w_2x + b_2) + b_3$$

$$\text{Output} = (w_3w_1 + w_4w_2)x + (w_3b_1 + w_4b_2 + b_3)$$

- Even with multiple neurons, the result is still in the form $w^*x + b$
- Because this is still a linear equation, it cannot model the curves found in real-world data like the delivery time example

The Limitation of Linear Stacking

- Even with multiple neurons in a hidden layer, the combined output remains a linear equation
 - If every operation is just multiplying by weights and adding biases, the result is mathematically equivalent to a single linear transformation ($W * x + b$)



$$\text{Output} = w_3(w_1x + b_1) + w_4(w_2x + b_2) + b_3$$

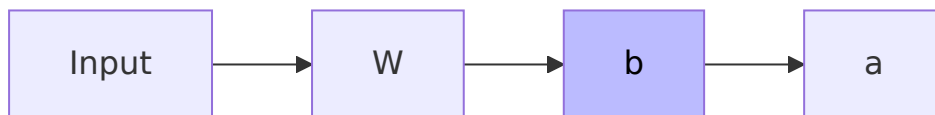
$$\text{Output} = (w_3w_1 + w_4w_2)x + (w_3b_1 + w_4b_2 + b_3)$$

$$\text{Output} = W^*x + b$$

- **[Conclusion]** This is still just a straight line

Activation Functions

- **[The Solution]** Non-linear activation functions
 - These add the necessary complexity to allow the model to learn curves and interesting patterns
 - In frameworks like PyTorch, these are applied element-wise to each neuron in a layer

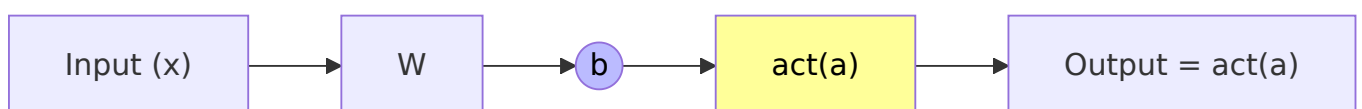


$$a = W * x + b$$

(Note: The activation function will be applied to this linear transformation to produce the final neuron output a)

Activation Functions

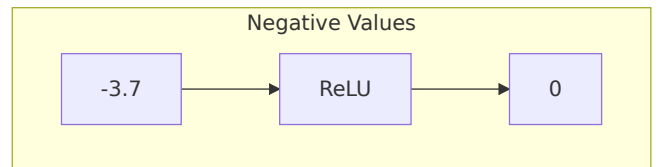
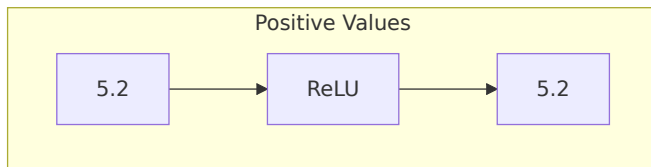
- **[The Purpose]** To allow a model to learn much richer, non-linear patterns
 - Instead of passing the raw linear transformation output directly, it is passed through an activation function



- The output of the linear transformation is $a = Wx + b$
- The final output of the neuron becomes $\text{act}(a)$

ReLU (Rectified Linear Unit)

- A simple and powerful activation function
- **[Logic]**
 - If the input is negative, the output is 0
 - If the input is positive, the output is the input itself (unchanged)



- In PyTorch, this is implemented via `nn.ReLU()`
- The logic can be summarized as:

```
if input > 0:
    return input
else:
    return 0
```

Implementing ReLU in PyTorch

- **[Implementation]** Adding ReLU to a model is a single-line operation in PyTorch
- Using `nn.Sequential` creates an "assembly line" where data flows through layers in order

```
# Define the model
model = nn.Sequential(
    nn.Linear(1, 1),
    nn.ReLU()
)
```

- In this setup, the data first passes through the `nn.Linear` layer, and then the output is transformed by the `nn.ReLU` layer

From Linear to Non-Linear

- **[The Transformation]** Adding ReLU allows a single neuron to model curves rather than just straight lines
- A single neuron starts with a linear transformation:

$$\text{Output} = W \times \text{distance} + b$$

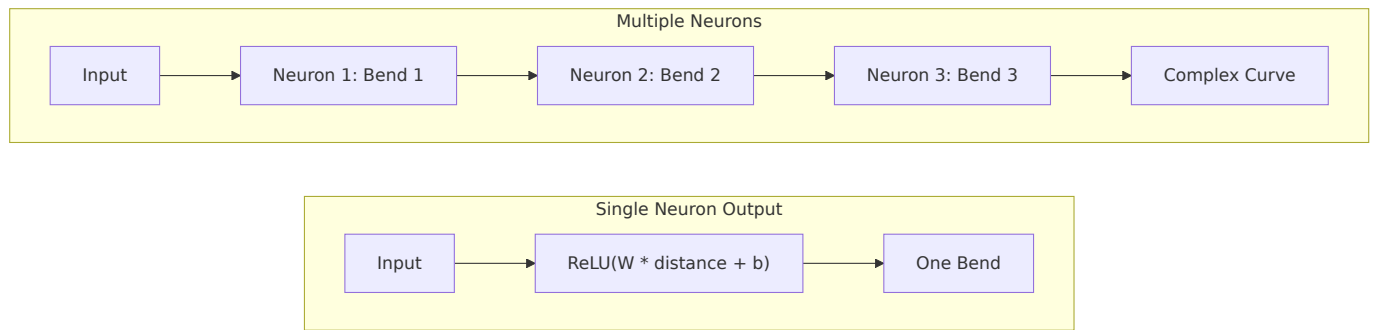
- By applying ReLU to this result, the model can capture non-linear relationships by "cutting off" the negative portion of the linear function.

The Non-Linearity of ReLU

- **[The Transformation]** Applying ReLU to a linear output creates a "bend" or "corner" in the function, breaking the straight-line pattern
 - When $Wx + \text{extdistance} + b$ is negative, the output is a flat line at 0
 - When the value is positive, the output follows the linear path normally
- **[The Bend Point]** The corner/bend in the graph occurs exactly where the linear transformation crosses zero
 - Mathematical condition for the bend: $Wx + b = 0$
 - Solving for x gives the location of the bend: $x = -b/W$

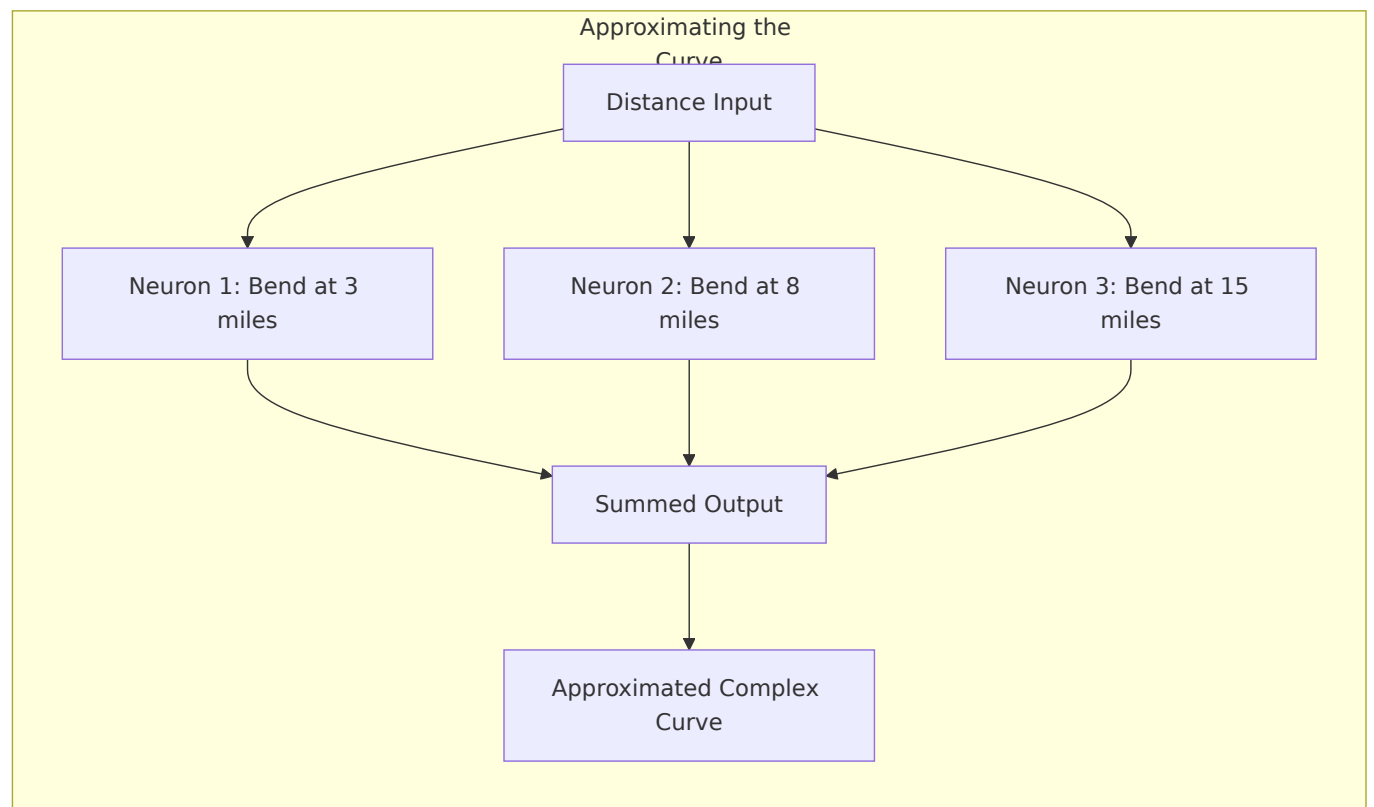
Modeling Curves with Multiple Neurons

- **[The Limitation of One Neuron]** A single neuron with ReLU only provides one bend in the function
 - This is insufficient for complex patterns like city transportation, where the relationship curves gradually as conditions change across various distances
- **[The Solution: Multiple Bends]** If one neuron provides one bend, then multiple neurons can provide multiple bends to approximate a curve
 - Each neuron brings its own unique parameters:
 - Weight (W)
 - Bias (b)
 - By combining these, the model can create a series of bends that collectively mimic a smooth, non-linear curve



Approximating Curves with Multiple Neurons

- **[Distributed Activation]** Because each neuron has its own unique weight and bias, they can be tuned to "activate" (start outputting a non-zero value) at different input thresholds
 - In the delivery time example, three neurons might be configured to respond to different stages of a trip:
 - **Neuron 1:** Activates around 3 miles (e.g., when city traffic begins)
 - **Neuron 2:** Activates around 8 miles (e.g., when entering highways)
 - **Neuron 3:** Activates around 15 miles (e.g., at full highway speeds)
- **[Summing the Outputs]** By adding the outputs of these neurons together, the individual "bends" combine to approximate a smooth, complex curve rather than a single sharp corner



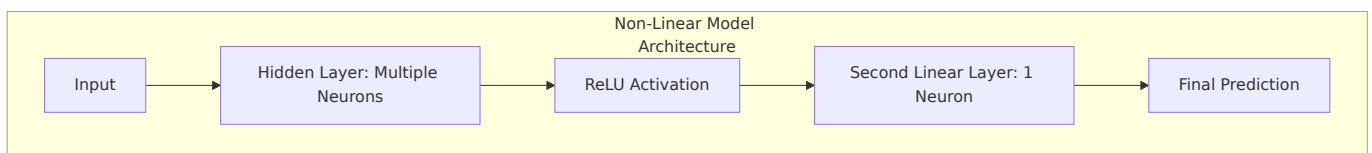
- **[Implementation Example]** A model with multiple neurons and non-linear activation can be defined in PyTorch using `nn.Sequential`:

```
model = nn.Sequential(  
    nn.Linear(1, 3),  
    nn.ReLU(),  
    nn.Linear(3, 1)  
)
```

- The `nn.Linear(1, 3)` layer creates 3 neurons, each taking 1 input
- `nn.ReLU()` applies the non-linearity to all 3 neurons
- `nn.Linear(3, 1)` sums their outputs into a single final prediction

The Final Prediction and Model Capacity

- **[The Aggregation Step]** After the hidden neurons produce their non-linear outputs via ReLU, a second linear layer combines them
 - This layer takes the multiple transformed values and reduces them to a single final prediction (e.g., the predicted delivery time)
- **[Increasing Complexity]** To achieve a smoother approximation of a curve, you can simply increase the number of neurons in the first layer
 - More neurons allow the model to learn more specific activation points and contribution strengths
 - With enough neurons, the network can approximate any smooth, complex curve, such as intricate city transportation patterns



- **[Scaling the Model]** Increasing the neuron count changes the dimensions in the PyTorch definition:

```
# Example of a model with 5 neurons instead of 3
model = nn.Sequential(
    nn.Linear(1, 5),
    nn.ReLU(),
    nn.Linear(5, 1)
)
```

- `nn.Linear(1, 5)`: The first layer now outputs 5 transformed values instead of 3

Alternative Activation Functions

- While ReLU is a common workhorse in deep learning due to its speed and effectiveness, other functions are used for specific mathematical requirements:
 - **Sigmoid**
 - Squashes outputs into a range between 0 and 1
 - Ideal for modeling probabilities
 - **TanH (Hyperbolic Tangent)**
 - Maps values to a range between -1 and 1
 - Useful for many different neural network tasks

Activation Function	Output Range	Primary Use Case
ReLU	$[0, \infty)$	General purpose; fast and effective workhorse
Sigmoid	$(0, 1)$	Modeling probabilities
TanH	$(-1, 1)$	Various neural network tasks

- **[Further Learning]** To dive deeper into these concepts, refer to:
 - PyTorch documentation (docs.pytorch.org)
 - DeepLearning.AI specializations

Moving from Theory to Practice

- **[Solving the Original Problem]** The non-linear model architecture discussed is the tool required to model the complex 'curve city' data that the initial linear model could not handle
 - By incorporating ReLU, the model can finally capture the intricate shifts in transportation patterns (e.g., transitioning from bike to car speeds)
- **[Next Steps]** The transition from understanding the mechanics to practical implementation involves applying these layers within a lab environment to conquer real-world non-linear datasets

Tensors

- **[Importance]** Mastering tensor operations is essential because many PyTorch errors arise from tensor-related issues
- **Core Concepts to Master**
 - Reading and understanding tensor shapes
 - Handling data types
 - Creating tensors in various ways
 - Reshaping and slicing tensors to fit model requirements
 - Extracting specific data from tensors

Checking Tensor Shapes

- Using the `.shape` attribute is the most important tool for verifying how data is organized
- **[Example]** Checking the shape of a delivery distance tensor:

```
# Your delivery data
distances = torch.tensor([[3.0], [7.0], [12.0], [18.0], [22.0],
[28.0]])
print(distances.shape)
```

```
- **Output:** `torch.Size([6, 1])`
```

- **Interpreting the output** `torch.Size([6, 1])`
 - **6**: The **batch size** (the number of samples in the dataset)
 - **1**: The number of **features** per sample (e.g., the distance of each delivery)

- **[Common Pitfalls]** Shape mismatches are one of the most frequent errors in PyTorch, alongside issues with device placement and data types (D-type)

Batch Size and the First Dimension

- The first dimension of a tensor represents the **batch size**
 - This is the number of individual samples the model will process at once
- **[Analogy]** Think of the batch as a stack of papers
 - The model reads each page (sample) the same way
 - Whether the stack has 6 pages or 600, the first dimension tells the model how many items are in that stack
- **[Example]** Passing a batch to a single-neuron model:

```
# distances shape is [6, 1]

# simple_model expects 1 feature per sample
output = simple_model(distances)
```

```
- Even though there are 6 samples, it works because the model's
  requirement for the number of features (the second dimension) is met
- The first dimension (`6`) does not cause a problem because the model
  is designed to iterate through the batch dimension
```

Tensor Shape Mismatches

- Errors occur when the number of features in the input tensor does not match the number of features the model was built to expect
- **[Example]** Passing multiple features (distance, hour, weather) to a model designed for one feature:


```
# Model expects 1 feature per sample
simple_model = nn.Linear(1, 1)

# Tensor provides 3 features per sample
distances = torch.tensor([[3.0, 7.0, 1.0], [18.0, 22.0, 2.0]])

output = simple_model(distances)
```

```
- **Error Output:** `RuntimeError: mat1 and mat2 shapes cannot be
multiplied (2x3 and 1x1)`
- In this error, `2x3` represents the input shape (2 samples, 3
features) and `1x1` represents the model's expected weight shape
```

- **[Debugging Tip]** PyTorch identifies *that* a mismatch happened, but not *how* to fix it; the solution usually becomes obvious once you compare the two shapes mentioned in the error message

Tensor Data Types (dtypes)

- PyTorch uses default data types based on the values provided during creation
- **Default Inference Rules**
 - **Integers:** Using whole numbers results in `torch.int64`
 - **Decimals:** Including a decimal point results in `torch.float32`

```
# Integer tensor
int_tensor = torch.tensor([1, 2, 3])
print(int_tensor.dtype)

# Output: torch.int64

# Float tensor
float_tensor = torch.tensor([1.0, 2.0, 3.0])
print(float_tensor.dtype)

# Output: torch.float32
```

Tensor Data Types

- PyTorch can infer dtypes automatically based on input values
 - Integers (e.g., 1, 2, 3) default to torch.int64
 - Decimals (e.g., 1.0, 2.0, 3.0) default to torch.float32
- **[Controlling Dtypes]** You can be explicit to ensure precision or avoid errors
 - Use the dtype argument during creation:

```
float_tensor = torch.tensor([1, 2, 3], dtype=torch.float32)
```

- Use the `.float()` method to convert an existing tensor to `'float32'`:

```
float_tensor = int_tensor.float()
```

- **[Type Promotion]** PyTorch automatically handles operations between different dtypes
 - Instead of throwing an error when mixing types, PyTorch promotes the result to a compatible type
 - **[Example]** Adding an integer tensor and a float tensor:

```
float_tensor = torch.tensor([1.0, 2.0, 3.0], dtype=torch.float32)
int_tensor = torch.tensor([1, 2, 3], dtype=torch.int64)
mixed_tensor = float_tensor + int_tensor

print(f"mixed type: {mixed_tensor.dtype}")
```

- The `mixed_tensor` will result in a `float32` type to accommodate the decimal precision

Common dtypes and Use Cases

- **[Type Promotion Details]** When mixing types, PyTorch automatically converts the less precise type to the more precise one (e.g., converting an `int` to a `float` during addition) to return a compatible result
- **Standard dtypes**
 - `float64`: Used when extra precision is required
 - `int8`: Used for memory efficiency
 - `float32`: The "sweet spot" for neural networks
 - It is fast, accurate, and the standard on modern hardware

Creating Tensors

- **Using Python Lists**
 - The simplest way to create a tensor is by passing a standard Python list to `torch.tensor()`
 - This behavior is very similar to how NumPy handles array creation

```
my_distances = [[3.0], [7.0], [12.0], [18.0]]
tensor_from_list = torch.tensor(my_distances)
```

- **[From Python Lists]** The simplest way to create a tensor is by passing a nested list to `torch.tensor()`

```
my_distances = [[3.0, 7.0], [12.0, 18.0]]
tensor_from_list = torch.tensor(my_distances)
```

- **[From NumPy Arrays]** You can convert existing NumPy arrays using `torch.from_numpy()`
 - **[Warning]** This method shares memory; modifying the NumPy array will also modify the tensor, and vice versa

```
import numpy as np
numpy_array = np.array([1.0, 2.0, 3.0])
tensor_from_numpy = torch.from_numpy(numpy_array)
```

- **[Built-in Patterns]** Useful for quickly generating test data:
 - `torch.zeros(rows, cols)` : Creates a tensor filled with 0s
 - `torch.ones(rows, cols)` : Creates a tensor filled with 1s
 - `torch.rand(rows, cols)` : Creates a tensor with random values

Reshaping Tensors

- Tensors often need to be reshaped to match a model's expected input dimensions
- **[The Batch Dimension Requirement]** PyTorch models expect inputs to include a batch dimension
 - For example, if a model expects a single feature, the input shape should be `[batch_size, 1]` (e.g., `6x1`) rather than just `[6]`
 - **[Common Error]** Forgetting this first dimension is one of the most frequent causes of shape mismatch errors in PyTorch

Manipulating Dimensions with Squeeze and Unsqueeze

- **[The Scalar Problem]** A single data point (like a scalar) does not meet the requirement for a `[batch_size, features]` shape
 - For example, a single distance of `25.0` is a scalar, but a model expects at least a `1x1` shape
- **Using `unsqueeze()`**
 - Adds a dimension of size 1 at a specific index
 - Useful for adding the required batch dimension to a single sample

```
# Your delivery data
single_distance = torch.tensor(25.0)

# Add batch dimension: torch.Size([1])
with_batch = single_distance.unsqueeze(0)

# Add feature dimension: torch.Size([1, 1])
ready_for_model = with_batch.unsqueeze(1)
```

- **Using `squeeze()`**
 - Removes all dimensions of size 1
 - Great for "cleaning up" after batching by returning the tensor to its original dimensionality

```
# Removes the extra dimensions added by unsqueeze
squeezed = ready_for_model.squeeze()

# Result: tensor(25.0)
```

- **[Debugging Tip]** Always use `print(tensor.shape)` when debugging to ensure your dimensions match what the model expects.

Indexing and Slicing

- Accessing specific values or subsets of a tensor works just like standard Python list indexing and slicing
 - `tensor[0]` retrieves the first element
 - `tensor[:3]` retrieves a slice (e.g., the first three elements)
- **[Important]** Even when you index into a tensor to get a single value, the result is still a tensor

Extracting Python Scalars

- Use the `.item()` method to convert a tensor containing exactly one element into a standard Python number (like a float)

- **[Constraint]** `.item()` can only be called on tensors with exactly one element; calling it on a larger tensor will result in an error

```
# Example of indexing and extracting a value
predictions = torch.tensor([[14.9], [24.1], [35.6], [45.2]])

# Get the first prediction (still a tensor)
first = predictions[0]

# Get the first three predictions (still a tensor)
first_three = predictions[:3]

# Convert a single-element tensor to a Python float
value = predictions[0].item()

# Output: 14.89999618530273 (class 'float')
```

Indexing Across Multiple Dimensions

- When working with multiple features per sample, you can index across both dimensions simultaneously
- **[Example]** Using the colon `:` operator allows you to select all elements in a specific dimension

```
# Data with multiple features: [distance, hour, weather]
data = torch.tensor([[3.0, 8.0, 1.0],
                    [7.0, 17.0, 2.0],
                    [12.0, 12.0, 1.0]])

# Extract only the 'distance' feature (the first column)

# The ':' selects all rows, and '0' selects the first column
distances = data[:, 0]

# Output: tensor([ 3.,  7., 12.])
```

Tensor Math and Broadcasting

Element-wise Operations

- PyTorch performs math operations element-wise
 - This means each element in a tensor is operated on independently
- **[Example]** Applying a single-neuron model ($\text{weight} \times \text{distance} + \text{bias}$) to multiple distances

```
distances = [[3.0], [7.0], [12.0]]
weight = 2.3
bias = 8.0

# The same weight and bias are applied to every element in the distances
# tensor
weight * distances + bias

# Resulting calculation per element:

# [2.3 * 3.0 + 8.0, 2.3 * 7.0 + 8.0, 2.3 * 12.0 + 8.0]
```

Efficiency and Compatibility

- PyTorch math syntax is similar to regular Python, but operations are executed efficiently across all elements simultaneously
- Supported data types for math operations:
 - **Scalars:** Single values
 - **Tensors with the same shape:** Operations are performed element-wise between tensors of identical dimensions

[Example] Multiplying two tensors of the same shape:

```
[[1.0, 2.0, 3.0]] * [[3.0, 7.0, 12.0]] = [[1.0*3.0, 2.0*7.0, 3.0*12.0]]
```

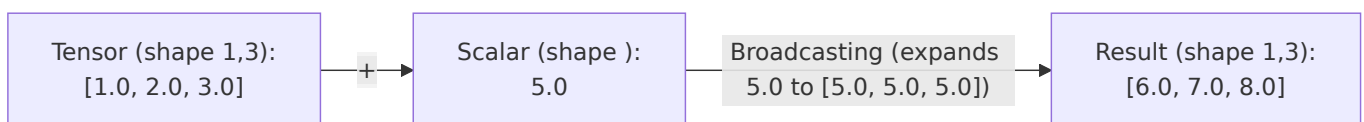
Applying Adjustment Factors to Complex Data

- When dealing with multi-feature data, different factors can be applied to different features
- **[Example]** Adjusting delivery features (distance, hour, weather) using specific multipliers:
 - Distance: 1.1x factor
 - Hour: 1.0x (no change)
 - Weather: 5.0x penalty

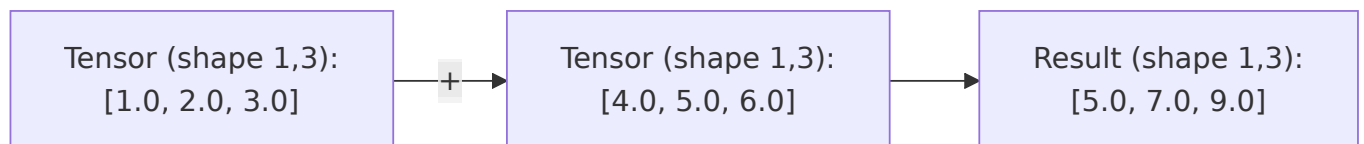
```
# Applying specific adjustment factors to a feature tensor
torch.tensor([[3.0, 8.0, 1.0], [7.0, 17.0, 2.0], [12.0, 12.0, 1.0]]) *
torch.tensor([[1.1, 1.0, 5.0], [1.1, 1.0, 5.0], [1.1, 1.0, 5.0]])
```

Broadcasting

- A mechanism that allows PyTorch to perform operations on tensors of different shapes
- **[How it works]** PyTorch automatically expands a smaller tensor (or a scalar) to match the shape of a larger tensor during an operation
 - This eliminates the need to manually create redundant tensors with repeated values
- **[Example]** Adding a scalar to a tensor:
 - The scalar is automatically expanded to match every element in the tensor

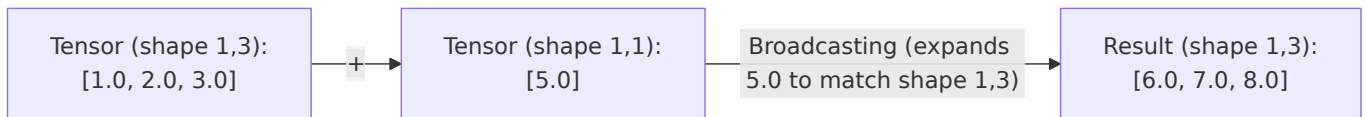


- **[Example]** Adding two tensors of the same shape (no broadcasting required):



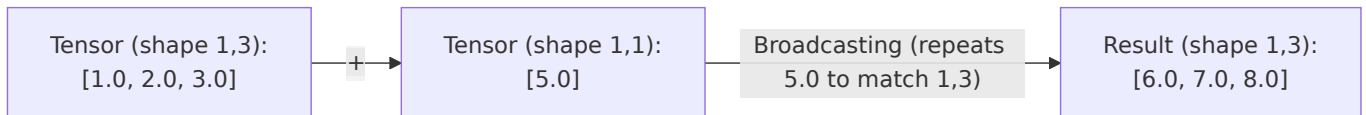
Broadcasting with Mismatched Shapes

- PyTorch can handle operations between tensors that do not have exact dimension matches through broadcasting
- **[Example]** Operating on tensors with different shapes:
 - Tensor 1 shape: (1,3) (e.g., `[[1.0, 2.0, 3.0]]`)
 - Tensor 2 shape: (1,1) (e.g., `[[5.0]]`)



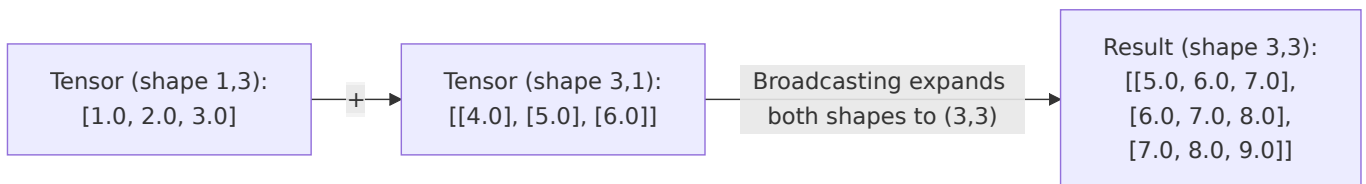
Expanding Dimensions via Repetition

- If one dimension is 1 and the other is larger, PyTorch automatically expands the smaller dimension by repeating its values
 - **[Example]** Adding a (1,1) tensor to a (1,3) tensor:



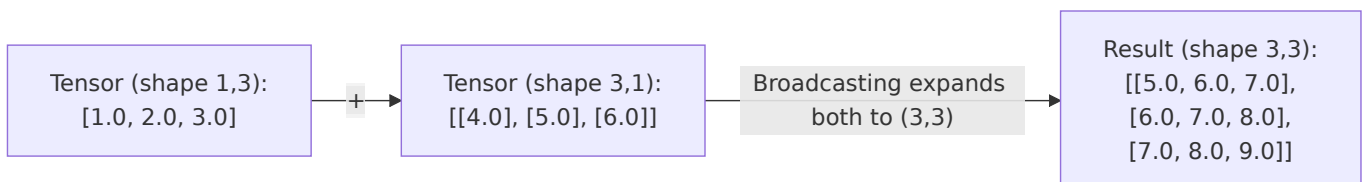
Complex Broadcasting: Combining Row and Column Vectors

- Broadcasting can occur across multiple dimensions simultaneously
- **[Example]** Combining a row vector (1,3) with a column vector (3,1):
 - PyTorch looks at each dimension and expands them to match



Broadcasting Dimension Expansion

- When combining a row vector (1,3) and a column vector (3,1), PyTorch expands both to a (3,3) shape
 - The first dimension (1 vs 3) expands the 1 to 3
 - The second dimension (3 vs 1) expands the 1 to 3



Practical Benefits of Broadcasting

- **[Why use it?]** To avoid manual repetition or using loops when applying adjustments
 - Instead of manually repeating values to match shapes, broadcasting handles the expansion automatically

- This pattern is used everywhere, such as adjusting multiple features across batches

```
# Instead of manual repetition, PyTorch handles expansion automatically
torch.tensor([[3.0, 8.0, 1.0], [7.0, 17.0, 2.0], [12.0, 12.0, 1.0]]) *
torch.tensor([[1.1, 1.0, 5.0], [1.1, 1.0, 5.0], [1.1, 1.0, 5.0]])
```

Applications of Broadcasting

- Broadcasting is a ubiquitous pattern in deep learning
 - It is used to apply transformations efficiently across different dimensions of data
 - It eliminates the need for manual loops or repetitive data structures when adjusting features or batches

Module 1 Wrap-up and Next Steps

- **Accomplishments in Module 1:**
 - Learned the fundamentals of PyTorch
 - Worked through the machine learning pipeline
 - Trained a first neural network
 - Mastered tensor operations
- **Immediate Next Steps:**
 - **Tensor Lab:** Practical application of the concepts learned
 - **Graded Assignment:** Assessment of skills
- **Module 2 Preview:**
 - Tackling classification problems
 - Diving deeper into how neural networks learn